Suman Educational Trust's

Dilkap Research Institute of Engineering & Management Studies

Mamdapur, Post:Neral, Taluka:Karjat, Dist:Raigad-410101

Department of Computer Science & Engineering

(Internet of Things & Cyber Security including Blockchain Technology)

A

Mini Project Report On

"MediGuide AI – AI-Powered Medicine Scanner"

Submitted in partial fulfillment of the requirement of the University of Mumbai for the Degree

Of

Bachelor of Engineering

(Computer Science & Engineering - Internet of Things

& Cyber Security including Blockchain Technology)

By

Group member 1

Group member 2

Group member 3

Group member 4

Under the guidance of

Prof. (Guide name)

University of Mumbai

Academic Year 2025-26

Dilkap Research Institute of Engineering &
Management Studies
Department of Computer Science & Engineering (Internet of
Things & Cyber Security including Blockchain Technology)
Academic Year 2025-26

CERTIFICATE

This is to certify that Group member 1, Group member 2, Group member 3, Group member 4 Of Sem IV, S.E. Computer Science & Engineering-Internet of Things & Cyber Security including Blockchain Technology, has satisfactorily completed the requirements of the Mini-Project entitled

Prof. (Guide name)	**Prof. Akshata Laddha**	
Guide	**HOD**	**External Examiner**

"MediGuide AI – AI-Powered Medicine Scanner"

As prescribed by the University of Mumbai Under the guidance of

Prof.(Guide name)

Guide

(Prof.

)

Internal Examiner

HOD

(Prof. Akshata Laddha)

External Examiner

Table of Contents

Abstract 4

Acknowledgements 5

List of Abbreviations 6

List of Figures 6

List of Tables 7

1. Introduction. 8

1.1 Introduction. 8

1.2 Motivation 8

1.3 Problem Statement & Objectives 9

1.4 Organization of the report 10

2. Literature Survey. 11

2.1 Survey of existing system 11

2.2 Limitation Existing system or research gap 12

2.3 Mini Project Contribution 12

3. Proposed System 13

3.1 Introduction 13

3.2 Architecture / Framework 14

3.3 Algorithm and process design 15

3.4 Details of hardware and software. 18

4 Experiment and Results 19

5 Conclusion and Future work 21

Abstract

"MediGuide AI – AI-Powered Medicine Scanner" is a modern web application developed using

React with TypeScript for the frontend and the Fireworks AI Vision API as the backend intelligence

layer. This application offers a platform where users can scan or upload images of medicines and

receive comprehensive, AI-generated information including medicine name, type, dosage, timing,

uses, side effects, drug interactions, storage instructions, and warnings. The app supports

multilingual output in 19 languages including Hindi, Marathi, Urdu, Spanish, French, Arabic,

Chinese, Japanese, Korean, and more. The purpose of this project is to provide an accessible,

user-friendly tool that empowers users—especially those with limited medical knowledge—to quickly

understand the medicines they are taking or considering. The application is built with a responsive,

mobile-first design using Tailwind CSS and Shadcn UI components, ensuring a seamless experience

across all devices. This report outlines the app's features, architecture, implementation process,

and the outcomes achieved.

Acknowledgement

We would like to express our gratitude and appreciation to all those who gave us the possibility

to complete this project and this report. I and my team thank our head of department Prof.

Akshata Laddha for giving us the necessary environment to acquire knowledge and skill. A

special thank-you to Prof. (guide name), whose help, stimulating suggestions and

encouragement, helped us to coordinate our project especially in writing this report.

We would also like to acknowledge with much appreciation the crucial role of the staff of

Information Technology Laboratory, who gave the permission to use all required machinery and

the necessary material to complete the report special thanks goes to our friends, who gave

suggestions about the project.

List of Abbreviations

AI Artificial Intelligence

API Application Programming Interface

UI User Interface

CSS Cascading Style Sheets

HTML HyperText Markup Language

JSON JavaScript Object Notation

VLM Vision Language Model

REST Representational State Transfer

SPA Single Page Application

HMR Hot Module Replacement

DOM Document Object Model

IDE Integrated Development Environment

NPM Node Package Manager

CRUD Create, Read, Update, Delete

List of Figures

Figure 1 System Architecture Diagram of MediGuide AI

Figure 2 Process Flowchart – Medicine Analysis Workflow

Figure 3 Component Hierarchy Diagram

Figure 4 MediGuide AI – Home Screen (Upload Interface)

Figure 5 MediGuide AI – Analysis Results Screen

Figure 6 MediGuide AI – Language Selection Interface

List of Tables

Table 1 Comparison of Existing Medicine Identification Apps

Table 2 MediGuide AI Feature Summary

Table 3 Hardware and Software Requirements

Introduction

1.1 Introduction:

MediGuide AI is an AI-powered medicine scanner web application designed to help users identify

medicines from images and retrieve structured, detailed health information instantly. With a clean,

mobile-first user interface, the app enables users to either take a photo using their device camera

or upload an existing image from their gallery. The captured image is then sent to the Fireworks AI

Vision Language Model (VLM), which uses advanced computer vision and natural language processing

to identify the medicine and return comprehensive details in a structured JSON format.

The application provides the following information for each scanned medicine:

• Medicine name (generic and brand name)

• Medicine type/category (e.g., antibiotic, analgesic, antacid)

• Recommended dosage information

• Timing instructions (before/after meals, morning/night)

• Medical uses and conditions treated

• Common side effects

• Drug interactions and contraindications

• Storage instructions

• Important warnings and precautions

A standout feature is its multilingual support, allowing users to receive all analysis results in

any of 19 supported languages, making it accessible to a diverse, global user base.

1.2

Motivation:

The motivation for developing MediGuide AI stemmed from a real-world observation: many people,

especially in developing regions, struggle to understand the medications they are prescribed or

purchase over the counter. Key motivating factors include:

• Language barriers: Medicine packaging is often printed in English or a single language, making

it inaccessible to non-native speakers.

• Lack of medical literacy: Many users cannot interpret dosage instructions, side effects, or drug interactions on their own.

• Inconvenience of manual lookup: Searching for medicine information online requires knowing the exact name, which may be difficult to read from packaging.

• Advancements in AI vision: Recent breakthroughs in Vision Language Models (VLMs) make it possible to accurately identify medicines from images and extract meaningful information.

These factors motivated us to create a tool that leverages AI to bridge the gap between medicine packaging and patient understanding, all through a simple camera scan.

1.3 Problem statement & Objectives

Problem Statement:

Patients and caregivers often face difficulty in understanding medicine information due to complex medical terminology, language barriers, and the absence of accessible digital tools that can instantly decode medicine packaging. Existing solutions either require manual text input, are region-specific, demand user registration, or lack multilingual support. There is a need for a simple, intelligent, and universally accessible tool that can identify medicines from images and present critical health information in the user's preferred language.

Objectives:

1. To develop a web application that uses AI-based computer vision to identify medicines from uploaded or captured images.

2. To provide structured and comprehensive medicine information including dosage, uses, side effects, interactions, storage, and warnings.

3. To implement multilingual output support for 19 languages to serve a diverse user base.

4. To create a responsive, mobile-first interface that works seamlessly across all devices

without requiring user registration or login.

5. To integrate a state-of-the-art Vision Language Model (Fireworks AI – Qwen3-VL-30B)

for accurate medicine identification and information extraction.

1.4 Organization of the Report

This report is organized as follows:

• Chapter 2 presents the literature survey, including existing systems and identified gaps.

• Chapter 3 describes the proposed system, including its architecture, algorithm design,

and technology stack.

• Chapter 4 outlines the experiments conducted and the results obtained.

• Chapter 5 concludes the report and suggests potential future work.

Literature Survey

2.1. Survey of Existing System

A detailed survey was conducted to analyze existing medicine identification and information

applications and identify their strengths and shortcomings:

1. Google Lens:

Google Lens can identify objects from images, including medicine packaging. However, it

provides generic search results rather than structured medical information. It does not offer

organized details like dosage, side effects, or drug interactions in a dedicated format.

2. Medscape / WebMD:

These platforms provide comprehensive drug information databases. However, they require

users to manually type the medicine name, which assumes the user can read the packaging.

They lack image-based identification capabilities.

3. Pill Identifier Apps (e.g., Drugs.com Pill Identifier):

These apps allow users to identify pills by shape, color, and imprint. However, they are limited to pill identification and do not work with medicine boxes, strips, or bottles. They are also region-specific (primarily US-focused).

4. Epocrates:

A professional-grade drug reference app used by healthcare providers. While comprehensive,

it requires registration, is designed for medical professionals, and lacks image-based scanning capabilities.

Table 1: Comparison of Existing Medicine Identification Apps

Feature	Google Lens	Medscape/WebMD	Pill ID Apps	MediGuide AI
Image-based scan	Yes	No	Partial	Yes
Structured output	No	Yes	Limited	Yes
Multilingual	Partial	No	No	Yes (19 langs)
No registration	Yes	No	Varies	Yes
Mobile responsive	Yes	Yes	Yes	Yes
AI-powered analysis	Limited	No	No	Yes

2.2. Limitations of Existing Systems or Research Gap

• Image-Based Identification: Most existing apps do not support scanning medicine images directly. Users must manually type the medicine name to look up information.

• Structured Medical Output: Apps like Google Lens return generic search results rather than

organized, structured information about dosage, side effects, interactions, etc.

• Multilingual Limitations: Very few existing systems offer medicine information in multiple languages, creating accessibility barriers for non-English-speaking populations.

• Registration Barriers: Many professional-grade drug reference apps require user registration,

creating friction for users who need quick, one-time lookups.

• Limited Medicine Form Support: Pill identifier apps work only with individual pills and cannot

identify medicines from boxes, strips, bottles, or syrup packaging.

• Lack of AI Vision Integration: Most existing systems rely on text-based search or manual

attribute matching. Very few leverage modern Vision Language Models for intelligent,

context-aware medicine identification.

2.3.

Mini Project Contribution

MediGuide AI addresses the identified gaps by:

• Providing AI-powered image-based medicine identification using a state-of-the-art Vision

Language Model (Qwen3-VL-30B via Fireworks AI).

• Delivering structured, comprehensive output covering 9 critical information categories

(name, type, dosage, timing, uses, side effects, interactions, storage, warnings).

• Supporting 19 languages for output, making it accessible to a global and multilingual audience.

• Offering a registration-free, instant-access experience with no login barriers.

• Working with all medicine forms (pills, strips, boxes, bottles, syrups) through flexible

image capture capability.

• Providing a mobile-first, responsive design ensuring usability across all device types.

Proposed System

3.1 Introduction

The proposed system, MediGuide AI, is a web application developed using React with TypeScript

and powered by the Fireworks AI Vision API. The application follows a Single Page Application

(SPA) architecture built on Vite for fast development and optimized production builds.

The system workflow is straightforward:

1. The user opens the web app on any device (mobile or desktop).

2. The user either takes a photo using their device camera or uploads an existing image.

3. The image is converted to Base64 format on the client side.

4. The Base64 image, along with a structured prompt and the selected output language, is sent

to the Fireworks AI Vision Language Model API.

5. The AI model analyzes the image and returns structured JSON containing medicine information.

6. The application parses the JSON response and displays the results in an organized,

card-based layout across 7 information sections.

7. The user can scan another medicine by resetting the interface.

Key design principles:

• Zero backend dependency: The app communicates directly with the Fireworks AI API from

the client, eliminating the need for a custom backend server.

• Privacy-focused: No images are stored; processing happens in real-time.

• Offline-ready architecture: The app shell loads instantly and only requires internet

connectivity for the AI analysis step.

3.2. Architecture/Framework

The application follows a component-based, client-side architecture with the following layers:

1. Presentation Layer (React + TypeScript):

• Built using React 18 with TypeScript for type safety.

• Uses Shadcn UI (built on Radix UI primitives) for accessible, customizable components.

• Styled with Tailwind CSS for utility-first, responsive design.

• Icon library: Lucide React for consistent, beautiful iconography.

2. State Management Layer:

• Local state management using React's useState hooks.

• Custom hook (useMedicineAnalysis) encapsulates all analysis logic, state, and error handling.

• TanStack Query (React Query) available for future data-fetching needs.

3. AI Integration Layer:

• Direct REST API calls to Fireworks AI inference endpoint.

• Model: Qwen3-VL-30B-A3B-Instruct (Vision Language Model).

• Communication via fetch API with JSON request/response format.

4. Build & Development Layer:

• Vite as the build tool with SWC for fast TypeScript/JSX compilation.

• PostCSS + Autoprefixer for CSS processing.

• ESLint for code quality enforcement.

Component Architecture:

■■■

■ ■ ■■■■■■■■■■■■■■■■■■■■■■ ■ ■

■ ■ ■ ImageUploader ■ (Camera/Upload)■ ■

■ ■ ■■■■■■■■■■■■■■■■■■■■■■ ■ ■

■ ■ ■LanguageSelector ■ (19 languages) ■ ■

■ ■ ■■■■■■■■■■■■■■■■■■■■■■ ■ ■

■ ■ ■ QuickTips ■ (Usage hints) ■ ■

■ ■ ■■■■■■■■■■■■■■■■■■■■■■ ■ ■

■ ■ ■ LoadingState ■ (Analysis) ■ ■

■ ■ ■■■■■■■■■■■■■■■■■■■■■■ ■ ■

■ ■ ■ MedicineResults ■ (7 sections) ■ ■

■ ■ ■■■■■■■■■■■■■■■■■■■■■■ ■ ■

■ ■■■■■■■■■■■■■■■■■■■

2.1 User selects output language from dropdown (default: English)

2.2 User clicks "Analyze Medicine" button

2.3 System enters loading state with animated indicators

Step 3: AI Vision Analysis

3.1 Extract raw Base64 data from the data URL

3.2 Construct structured prompt with language instruction:

- Prompt instructs the AI model to act as a medical information assistant
- Specifies 9 required output fields (name, type, dosage, timing, uses, sideEffects, interactions, storage, warnings)
- Requires output in the user's selected language
- Demands strict JSON format response

3.3 Send POST request to Fireworks AI API:

- Endpoint: https://api.fireworks.ai/inference/v1/chat/completions
- Model: accounts/fireworks/models/qwen3-vl-30b-a3b-instruct
- Payload: multimodal message with text prompt + Base64 image
- Parameters: max_tokens=2048, temperature=0.3 (low for factual accuracy)

Step 4: Response Processing

4.1 Validate API response status (handle errors)

4.2 Extract content from response: data.choices[0].message.content

4.3 Parse JSON from response:

- Attempt regex extraction: match pattern {..."name"..."warnings"...} (handles cases where model wraps JSON in thinking tokens or markdown)
- Fallback: direct JSON.parse on full content

4.4 Store parsed MedicineInfo object in state

Step 5: Results Display

5.1 Render 7 information cards with color-coded categories:

- Medicine Name & Type (peach background)
- Dosage & Timing (mint background)
- Uses (secondary background)
- Side Effects (destructive/red background)
- Drug Interactions (peach background)
- Storage Instructions (mint background)
- Warnings & Precautions (destructive/red background)

5.2 Display within scrollable area for mobile usability

5.3 Provide "Scan Another Medicine" button to reset

Algorithm 2: Error Handling & Recovery

Step 1: API Key Validation

- If VITE_FIREWORKS_API_KEY is not configured → display configuration error toast

Step 2: API Error Handling

- If API returns non-200 status → throw descriptive error with status text
- If no content in response → throw "No response from AI model"

Step 3: JSON Parse Error Recovery

- If primary regex extraction fails → attempt direct JSON.parse
- If all parsing fails → throw "Could not parse medicine information"

Step 4: User Notification

- Success: Green toast notification with identified medicine name
- Failure: Red destructive toast with error description

Process Flow Diagram:

User → [Take Photo / Upload Image] → [Base64 Conversion] → [Language Selection]
→ [Analyze Medicine Click] → [Loading Animation] → [Fireworks AI API Call]
→ [JSON Response Parsing] → [Results Display in 7 Cards] → [Scan Another / Reset]

Source Code Structure:

src/

■■■ App.tsx # Root component with providers & routing

■■■ main.tsx # Application entry point

■■■ index.css # Global styles & Tailwind directives

■■■ App.css # App-level styles

■■■ components/

■ ■■■ MedicineScanner/

■ ■ ■■■ ImageUploader.tsx # Camera/gallery image capture component

■ ■ ■■■ LanguageSelector.tsx # 19-language dropdown selector

■ ■ ■■■ LoadingState.tsx # Animated loading indicator

■ ■ ■■■ MedicineResults.tsx # 7-section results card display

■ ■ ■■■ QuickTips.tsx # Usage tips component

■ ■■■ NavLink.tsx # Navigation link component

■ ■■■ ui/ # Shadcn UI component library

■■■ hooks/

■ ■■■ useMedicineAnalysis.ts # Core analysis logic & state management

■ ■■■ use-mobile.tsx # Mobile detection hook

■ ■■■ use-toast.ts # Toast notification hook

■■■ lib/

■ ■■■ utils.ts # Utility functions (cn class merger)

■■■ pages/

■ ■■■ Index.tsx # Main application page

■ ■■■ NotFound.tsx # 404 error page

■■■ integrations/ # External service integration configs

3.4. Details of Hardware and Software:

Hardware Requirements:

• A computer with at least 4GB RAM for development

• Any modern smartphone, tablet, or computer with a web browser for usage

• Device camera (optional, for live photo capture)

• Internet connection (required for AI analysis API calls)

Software Requirements:

• React 18.3.1: Frontend JavaScript library for building user interfaces

• TypeScript 5.8.3: Typed superset of JavaScript for enhanced code quality

• Vite 5.4.19: Next-generation build tool with fast HMR

• Tailwind CSS 3.4.17: Utility-first CSS framework for responsive design

• Shadcn UI: Accessible component library built on Radix UI primitives

• Radix UI: Low-level UI primitives for accessible components

• Lucide React 0.462.0: Beautiful & consistent icon library

• TanStack React Query 5.83.0: Data fetching and state management

• React Router DOM 6.30.1: Client-side routing for SPA navigation

• Sonner 1.7.4: Toast notification library

• Zod 3.25.76: Schema validation library

• React Hook Form 7.61.1: Performant form handling

• Fireworks AI API: Vision Language Model inference endpoint

(Model: Qwen3-VL-30B-A3B-Instruct)

• Node.js & NPM: Runtime and package manager for development

• Visual Studio Code: IDE for coding and debugging

• Git: Version control system

• ESLint 9.32.0: Code linting and quality enforcement

Table 3: Hardware and Software Requirements Summary

Category	Requirement
RAM	Minimum 4GB (development), Any (usage)
Browser	Chrome, Firefox, Safari, Edge (modern versions)
Internet	Required for AI API calls
OS (Development)	Windows / macOS / Linux
OS (Usage)	Any OS with a modern web browser
Framework	React 18 + TypeScript + Vite
Styling	Tailwind CSS + Shadcn UI
AI Model	Qwen3-VL-30B via Fireworks AI
Version Control	Git + GitHub

4. Experiment and Results

During the development phase, the application was tested extensively for functionality, responsiveness, accuracy, and performance:

4.1 Functionality Testing:

• Image Upload: Verified that both camera capture and gallery upload correctly read image

files and convert them to Base64 format across multiple browsers (Chrome, Firefox, Safari).

• File Validation: Confirmed that non-image files are rejected and only supported formats

(PNG, JPEG, WebP, HEIC, HEIF) are accepted.

• API Integration: Tested the Fireworks AI API integration with various medicine images

including tablets, capsules, syrups, and medicine boxes.

• JSON Parsing: Verified that the regex-based JSON extraction correctly handles edge cases

including AI responses wrapped in thinking tokens or markdown code blocks.

• Language Output: Tested multilingual output across all 19 supported languages to ensure

accurate translation of medicine information.

• Error Handling: Simulated error conditions (invalid API key, network failure, unrecognizable

images) and confirmed appropriate error messages are displayed via toast notifications.

4.2 Usability Testing:

• Mobile Responsiveness: Tested on various mobile devices and screen sizes to ensure the

mobile-first design works correctly.

• Camera Integration: Verified that the capture="environment" attribute correctly triggers

the rear camera on mobile devices for medicine photography.

• Navigation Flow: Confirmed intuitive flow from image capture → analysis → results → reset.

• Accessibility: Ensured all interactive elements have appropriate ARIA labels and keyboard

navigation support through Radix UI primitives.

4.3 Performance Testing:

• Load Time: The application shell loads in under 2 seconds on standard connections due to

Vite's optimized build output.

• API Response Time: The Fireworks AI analysis typically completes in 3-8 seconds depending

on image complexity and network conditions.

• Client-Side Processing: Base64 conversion and JSON parsing execute efficiently with negligible client-side overhead.

Results:

• The AI model accurately identified medicines from clear images of tablets, strips, boxes, and syrup bottles with high confidence.

• Medicine information was returned in the correct structured format across all tested scenarios.

• Multilingual output was verified for Hindi, Marathi, Spanish, French, Arabic, Chinese, Japanese, and Korean with contextually accurate translations.

• The application performed smoothly on mobile devices with no perceivable lag during navigation and interaction.

• Error recovery worked as expected, with descriptive toast notifications guiding users to resolve issues.

Table 2: MediGuide AI Feature Summary & Test Results

Feature	Status	Notes
Camera Capture	Working	Rear camera on mobile
Gallery Upload	Working	Supports PNG, JPEG, WebP, HEIC
AI Medicine Identification	Working	Qwen3-VL-30B model
Structured JSON Output	Working	9 information fields
Multilingual Support	Working (19 langs)	All languages verified
Responsive Design	Working	Mobile-first approach
Error Handling	Working	Toast notifications
Loading Animations	Working	Smooth pulse & bounce effects
No Login Required	Yes	Zero registration barriers

5. Conclusion and Future Work

Conclusion:

MediGuide AI successfully meets its objectives by providing an intelligent, user-friendly platform

for scanning medicine images and receiving comprehensive, structured health information in the

user's preferred language. The application demonstrates effective use of modern web technologies

(React, TypeScript, Vite, Tailwind CSS) for frontend development and a state-of-the-art Vision

Language Model (Qwen3-VL-30B via Fireworks AI) for AI-powered medicine identification.

Key achievements of the project include:

• Successfully integrated computer vision AI for real-time medicine identification from images.

• Delivered structured output across 9 critical information categories.

• Implemented multilingual support for 19 languages, making the tool globally accessible.

• Created a clean, responsive, mobile-first interface with zero registration barriers.

• Achieved reliable performance with accurate results across diverse medicine forms and packaging.

The project demonstrates the practical application of Vision Language Models in healthcare

accessibility, showing how AI can bridge the gap between complex medical packaging and patient

understanding.

Future Work:

• Offline Mode: Implementing a local AI model or caching mechanism to allow basic medicine

lookup without internet connectivity using service workers.

• Medicine History: Adding optional local storage to maintain a history of scanned medicines

for user reference.

• Drug Interaction Checker: Building a feature where users can scan multiple medicines and

check for potential interactions between them.

• Barcode/QR Scanning: Integrating barcode and QR code scanning for faster and more accurate

medicine identification.

• Voice Output: Adding text-to-speech functionality to read out medicine information for

visually impaired users or elderly users.

• PWA Support: Converting the application into a Progressive Web App for installable,

app-like experience with offline caching.

• Community Database: Building a crowd-sourced medicine database where verified AI results

can be stored for faster future lookups.

• Healthcare Provider Integration: Adding the ability to share medicine information directly

with healthcare providers or pharmacists.

References

[1] React Documentation - https://react.dev/

[2] TypeScript Handbook - https://www.typescriptlang.org/docs/

[3] Vite Build Tool - https://vitejs.dev/

[4] Tailwind CSS Documentation - https://tailwindcss.com/docs

[5] Shadcn UI Components - https://ui.shadcn.com/

[6] Radix UI Primitives - https://www.radix-ui.com/

[7] Fireworks AI API Documentation - https://docs.fireworks.ai/

[8] Qwen Vision Language Model - https://huggingface.co/Qwen

[9] Lucide React Icons - https://lucide.dev/

[10] TanStack React Query - https://tanstack.com/query/latest